

OOP 練習 2：方法與參數

學習目標

- 會定義方法的四種參數：IMPORTING / EXPORTING / CHANGING / RETURNING
- 理解 RETURNING 的價值：呼叫本身就是「值」，可以用 DATA(...) 直接接住、塞進 string template
- 能對照三種模組化寫法的呼叫差異：FORM (練習 8) / FM (練習 15) / Method
- 開始隨堂使用現代語法：inline declaration DATA(...)、string template |...{ }...|

事前準備

建立程式 ZR_0002_<你的姓名縮寫>，套件 \$TMP。練習 15 的 FM Z_TR15_CALC_REVENUE 會當對照組（系統上已存在，直接呼叫）。

題目需求

1. 定義 local class lcl_revenue，方法 calc：
 - IMPORTING iv_price TYPE s_price、iv_seatsocc TYPE s_seatsocc
 - RETURNING VALUE(rv_revenue) TYPE s_price (營收 = 票價 × 座位數)
2. 同類別再寫一個 calc_exp：輸入相同，但改用 EXPORTING ev_revenue 回傳——體會呼叫端寫法差多少
3. 再寫 apply_discount：IMPORTING iv_percent TYPE i、CHANGING cv_price TYPE s_price，把呼叫端的價格就地打折
4. 主程式做三段對照，輸出同一筆計算（票價 1500 × 200 座）：
 - CALL FUNCTION 'Z_TR15_CALC_REVENUE' (舊寫法)
 - calc_exp (方法版，但還是要先宣告變數接結果)
 - calc (用 DATA(...) 一行接住)，再加碼把 calc 直接塞進 string template 一行輸出
5. 最後示範 apply_discount：1000 元打 85 折

預期輸出 (範例)

```
FM 版      :      300,000.00
EXPORTING 版 :      300,000.00
RETURNING 版 :      300,000.00
一句話版本：營收 = 300000.00
1000 打 85 折 =      850.00
```

團隊實務備註

- 團隊風格：單一輸出一律用 RETURNING；真的有多個輸出才用 EXPORTING；CHANGING 盡量少用（讀程式的人難追蹤變數在哪被改）
- RETURNING 只能有一個，且必須 VALUE(...) (傳值)
- FM 的 EXCEPTIONS (classic exception) 在 OO 世界對應「例外類別」，op09 才教；本題的方法先不做防呆

- FM 呼叫的「方向反轉」(FM 定義的 IMPORTING = 呼叫端寫 EXPORTING) 在方法的完整呼叫式 (`calc_exp( EXPORTING ... IMPORTING ... )`) 仍然存在；但 RETURNING 的 **functional** 呼叫 (`go->calc( iv_price = ... )`) 完全不用寫方向關鍵字——這也是團隊偏好 RETURNING 的原因之一

思考題

1. 為什麼 RETURNING 限定一個、且必須傳值？想想 `a = go->calc( ... ) + go->calc( ... )`. 這種表達式。
2. `calc` 完全沒用到物件的任何屬性（無狀態），那一定要 NEW 出物件才能呼叫它嗎？（op04 靜態方法預告）
3. `apply_discount` 用 CHANGING 就地改價格，改成 RETURNING 回傳新價格會更好嗎？兩種設計各有什麼優缺點？

答案

見 `zr_oo02_method_params.prog.abap` (SAP 端程式 ZR_0002_METHOD_PARAMS)。